Lab 11 : Extending the CPU (SUB, Load Immediate, and Simple I/O)

Objectives

In this lab, you will extend the single-cycle CPU from Lab 10 in three ways: a subtract instruction (SUB), a load-immediate instruction (LDI), and two memory-mapped I/O ports (a 7-segment display and four switches). At the end, your CPU will run a small program that reads switches and shows values on a display.

- Add a subtract mode to miniALU and the control unit (SUB).
- Add load immediate (LDI) using the unused input of the write-back multiplexer.
- Build a memory-mapped output port (7-segment display) and input port (switches).
- Write and run programs that use the new instructions and ports.

Requirements

- Working cpu.dig, controlUnit.dig, miniALU.dig and the other .dig files from Labs 9 and 10.
- Copy all Lab 10 files into a new folder named Lab11 and work only on the copies.
- Digital simulator (same version as Labs 9 and 10).

Instruction Set After This Lab

Instruction format (14 bits, unchanged): Opcode(3) + Type_Bit(1) + Reg1_Addr(3) + Reg2_Addr(3) + Address_Field(4).

Opcode	Type	Instruction	Operation
000	**0**	LOAD Rd, Rs	Rd ← Rs
000	**1**	LOAD Rd, Mem[addr]	Rd ← Mem[addr] (addr 0 to 15)
001	**X**	STORE Rs, Mem[addr]	Mem[addr] ← Rs
010	**0**	ADD R1, R2	R1 ← R1 + R2, updates zero flag
010	**1**	SUB R1, R2 (new)	R1 ← R1 - R2, updates zero flag
011	**X**	BZ addr	PC ← addr if Zero_Flag = 1
100	**X**	BR addr	PC ← addr
101	**X**	LDI Rd, imm (new)	Rd ← Address_Field (0 to 15)
110, 111	**-**	unused	-

Memory map (Address_Field is 4 bits, so LOAD and STORE reach only Mem[0] to Mem[15]):

Address	Use	Access
Mem[0] to Mem[13]	Normal data RAM (variables)	LOAD, STORE
Mem[14]	7-segment display port	STORE only
Mem[15]	Switch port (SW3 to SW0)	LOAD only

Lab 11.1 : Subtract (SUB)

The ALU from Lab 9 can only add. You will make it subtract with the same adder, using A - B = A + (NOT B) + 1.

1. Modify miniALU

1.1. Open miniALU.dig. Add a 1-bit input labelled Sub.

1.2. Add a NOT gate (Data Bits = 4) and a 4-bit Multiplexer with 1 selector bit (2 inputs). Connect B to in_0 and NOT B to in_1. Connect Sub to sel.

1.3. Connect the multiplexer output to the B input of your adder, instead of connecting B directly.

1.4. Remove the Constant Value (0) on the carry-in (Ci) and connect Sub to Ci instead.

1.5. Test with the values below and record the Result:

A	B	Sub	Expected Result	Your Result
5	3	0	8	
5	3	1	2	
3	3	1	0 (NOR output = 1)	
2	5	1	13 (-3 in 4-bit two's complement)	

Q1: Why does inverting B and setting the carry-in to 1 give A - B?

2. Update controlUnit

2.1. Open controlUnit.dig. Add a 1-bit output ALUSub = ADD AND Type_Bit. (SUB uses the ADD opcode with Type_Bit = 1, so FlagWrite does not change and the zero flag is latched for SUB as well.)

2.2. In cpu.dig, reload miniALU (delete it and add it again if the Sub pin does not appear). Connect ALUSub from the controlUnit to miniALU.Sub.

Lab 11.2 : Load Immediate (LDI)

LDI copies the 4-bit Address_Field directly into a register, so constants no longer need to be preloaded into Data RAM.

1. Datapath

1.1. In cpu.dig, find the write-back multiplexer (the 4-input multiplexer with 2 selector bits whose output goes to registerFile.Din). Its in_3 is tied to a Constant Value (0). Delete that constant.

1.2. Connect the Address_Field output (4-bit, the top output pin of the primary splitter) to in_3. The same output also feeds Address_Field_Ext.

2. Control unit

2.1. Open controlUnit.dig. On the decoder, label the sixth output y5 = LDI (y0 to y4 are LOAD, STORE, ADD, BZ, BR).

2.2. Change the RegWrite OR gate to 3 inputs and connect y5: RegWrite = LOAD OR ADD OR LDI.

2.3. sel0 = (LOAD AND Type_Bit) OR LDI. Add a 2-input OR gate with the output of the existing AND gate and y5 as inputs. Connect its output to the top input of the RegWriteSource merger."

2.4. sel1 = (LOAD AND NOT Type_Bit) OR LDI. Add a second 2-input OR gate with the output of the existing AND gate and y5 as inputs. Connect its output to the bottom input of the RegWriteSource merger.

LDI therefore gives RegWriteSource = 11. LDI does not change FlagWrite, MemRead, MemWrite or PCSource.

3. Update the Test component in controlUnit.dig

Add the ALUSub column at the end and use these rows. All rows must pass.

```
Opcode Type Zero_Flag RegWrite MemRead MemWrite FlagWrite PCSource RegWriteSource ALUSub
0b000 0 0 1 0 0 0 0 0b10 0
0b000 1 0 1 1 0 0 0 0b01 0
0b001 0 0 0 0 1 0 0 0b00 0
0b010 0 0 1 0 0 1 0 0b00 0
0b010 1 0 1 0 0 1 0 0b00 1
0b011 0 0 0 0 0 0 0 0b00 0
0b011 0 1 0 0 0 0 1 0b00 0
0b100 0 0 0 0 0 0 1 0b00 0
0b100 0 1 0 0 0 0 1 0b00 0
0b101 0 0 1 0 0 0 0 0b11 0
0b101 1 0 1 0 0 0 0 0b11 0
0b110 0 0 0 0 0 0 0 0b00 0
0b111 0 0 0 0 0 0 0 0b00 0
```

4. Test program 1

Load the following into programROM (Edit Content):

Addr	Instruction	Encoding	Note
0	LDI R1, 5	**0x2885**	R1 ← 5
1	LDI R2, 3	**0x2903**	R2 ← 3
2	SUB R1, R2	**0x14A0**	R1 ← 2, ZF = 0
3	STORE R1, Mem[3]	**0x0883**	Mem[3] ← 2
4	SUB R1, R1	**0x1490**	R1 ← 0, ZF = 1
5	BZ 7	**0x1807**	Branch taken because ZF = 1
6	LDI R3, 9	**0x2989**	Must be skipped
7	BR 7	**0x2007**	Stop here (jumps to itself)

Single-step the program and verify: R1 = 0, R2 = 3, R3 = 0, Mem[3] = 2, and the PC stays at 7.

Q2: If BZ did not branch, what would R3 contain? What does this show about the zero flag?

TA Check: _____ Time: _____

Lab 11.3 : Output Port (7-Segment Display at Mem[14])

1. Address decode

1.1. In cpu.dig, add a Splitter/Merger (Input Splitting = 4, Output Splitting = 1,1,1,1). Connect the branched Address_Field wire to its input. The outputs are A0, A1, A2, A3 (A0 is the first output).

1.2. ADD a 3-input AND gate. IO_High = A3 AND A2 AND A1 (this is true for Mem[14] and Mem[15]).

1.3. Add a NOT gate. Connect A0 to its input. Label its output NOT_A0.

1.4. Add a 2-input AND gate. Connect IO_High and NOT_A0 to its inputs. Label its output DispSel. (True only for address 14.)

1.5. Add a second 2-input AND gate. Connect IO_High and A0 (not NOT_A0) to its inputs. Label its output SwSel. (True only for address 15. You will wire this to the switch input in Lab 11.4 — leave it unconnected downstream for now.)

1.6 Add a third 2-input AND gate. Connect MemWrite (from controlUnit) and DispSel to its inputs. Label its output DispWrite.

2. Output register

2.1. Add a Register (Data Bits = 4). Connect D to registerFile.Da, en to DispWrite, and C to the master clock.

2.2. Label the register's output DispValue (4-bit).

2.3 Note: dataRAM will also store whatever is written to Mem[14], since its str/ld/address inputs are not restricted by DispSel. This is harmless — the CPU only reads that location back through the switch/display ports, not through a normal LOAD — so you can ignore it.

3. Hex-to-7-segment decoder

3.1. Add a ROM (Address Bits = 4, Data Bits = 7). Connect sel to Supply voltage. Connect the output register DispValue to A.

3.2. Right-click the ROM, Edit Content. Enter these values for addresses 0 to F:

```
addr 0-7 :  3F 06 5B 4F 66 6D 7D 07
addr 8-F :  7F 6F 77 7C 39 5E 79 71
```

3.3. These are standard common-cathode hex-to-7-segment codes: bit 0 = segment a, bit 1 = b, bit 2 = c, bit 3 = d, bit 4 = e, bit 5 = f, bit 6 = g.

3.4. Add a Splitter/Merger configured as a splitter: Input Splitting = 7, Output Splitting = 1,1,1,1,1,1,1. Connect the ROM's D output to its input.3.5. Optional check: add a Seven-Segment Display (Hex) on the same 4-bit value and compare the two digits.

3.5 Add a Seven-Segment Display (Components → IO → Display → Seven-Segment Display). Connect the splitter's seven outputs, in order, to the display's a, b, c, d, e, f, g inputs (top 4 is a, b, c, d and bottom is e, f, g, dp). Connect dp to ground.

3.6 Optional check: add a Seven-Segment Display (Hex) (same menu) and connect it directly to DispValue. Run the simulation and confirm both displays always show the same digit — this isolates a decoder-table error from a wiring error.

4. Test program 2 (counts from 0 to F forever)

Addr	Instruction	Encoding	Note
0	LDI R1, 0	**0x2880**	R1 ← 0
1	LDI R2, 1	**0x2901**	R2 ← 1
2	STORE R1, Mem[14]	**0x088E**	Show R1 on the display
3	ADD R1, R2	**0x10A0**	R1 ← R1 + 1
4	BR 2	**0x2002**	Repeat

Q3: Which digits look the same as expected, and which (if any) look wrong? Fix the ROM contents if needed.

TA Check: _____ Time: _____

Lab 11.4 : Input Port (Switches at Mem[15])

1. Switch inputs

1.1. Add four Input components (1-bit) labelled SW3, SW2, SW1, SW0.

1.2 Merge them into a 4-bit bus named SW with a Splitter/Merger (Input Splitting = 1,1,1,1, Output Splitting = 4).

2. Input select multiplexer

2.1. Add a Multiplexer (Data Bits = 4, Selector Bits = 1). This gives two 4-bit inputs, in_0 and in_1, and a 1-bit sel.

2.2. Connect in_0 to dataRAM.D (the data-out pin of dataRAM).

2.3. Connect in_1 to SW (the 4-bit bus from step 1.2).

2.4. Connect sel to SwSel (built in Lab 11.3).

2.5. Label this multiplexer's output MemOrSwData (4-bit). When SwSel = 0 (any address except 15), the output equals dataRAM.D as before. When SwSel = 1 (address 15), the output equals the switch value instead.

3. Wire into the datapath

3.1. Find the write-back multiplexer (the same 4-input, 2-selector-bit multiplexer from Lab 11.2, whose output feeds registerFile.Din). Its in_1 currently connects directly to dataRAM.D (from Lab 10.1).

3.2. Delete the wire from dataRAM.D to in_1 of the write-back multiplexer.

3.3. Connect MemOrSwData (the output from step 2.5) to in_1 of the write-back multiplexer instead.

3.4. Nothing else about the write-back multiplexer changes: in_0 is still miniALU.Result, in_2 is still registerFile.Db, and in_3 is still Address_Field. A LOAD Rd, Mem[addr] instruction still selects in_1 via sel0/sel1 as before — it now just carries either the RAM value or the switch value, depending on addr.

4. Test program 3

Load into programROM:

Addr	Instruction	Encoding	Note
0	LOAD R1, Mem[15]	**0x048F**	R1 ← switches
1	STORE R1, Mem[14]	**0x088E**	Show R1 on the display
2	BR 0	**0x2000**	Repeat

5. Run and verify

5.1. Start the simulation. Single-step through addresses 0, 1, 2 repeatedly.

5.2. Between loops, toggle one or more of the four switch inputs (SW3, SW2, SW1, SW0) by double-clicking them. Try at least two different combinations, for example SW = 0101 (value 5) and SW = 1111 (value 15). The display must show the new switch value after each loop completes step 1 (the LOAD).

TA Check: _____ Time: _____

Lab 11.5 : Exercise - Countdown

Write a program that reads N (0 to 15) from the switches, then shows N, N-1, ..., 1, 0 on the display, one value per loop, and finally stops (BR to its own address). Use LDI for constants, SUB for the decrement, and BZ/BR for the loop.

Questions

Q4: How many instructions does your program contain?

Q5: For N = 3, how many clock cycles does your CPU need from address 0 until it first reaches the final stop instruction?

Q6: What happens when N = 0 in your program? If your counter wraps around, explain what you see using 4-bit two's complement.

Q7: Why can LDI only load values 0 to 15? Which instruction field supplies the value?

Q8: Mem[14] and Mem[15] no longer behave like normal memory. Explain memory-mapped I/O in your own words.

TA Check: _____ Time: _____

Submission

- miniALU.dig, controlUnit.dig, cpu.dig, and every other .dig file you changed (include the ROM and RAM contents you configured).
- Video record for Lab 11.5 Exercise
- Written answers to Q1 to Q8.

Submission at: